

universidad de León Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Metodología docente AI-SDLC

Francisco Javier Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías Industrial, Informática y Aeroespacial

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Explicar** el desplazamiento del esfuerzo y la hipótesis rectora.
- **Situar** el SDD y las cuatro modalidades de integración de la IA.
- **Describir** las diez fases del AI-SDLC y sus salidas.
- **Distinguir** autonomía de ejecución y autoridad de aprobación.
- **Reconocer** la gobernanza y los límites de la metodología.

Idea rectora y desplazamiento del esfuerzo

- La IA **reduce ciertos costes de implementación** (escritura, andamiaje, traducción, borradores de prueba) **sin eliminar** los problemas de correctitud, seguridad, mantenimiento, portabilidad y responsabilidad.
- La evidencia empírica disponible es contextualizada (METR 2025 midió una *ralentización* en su contexto) [1, 5].
- Lo que la IA hace es *desplazar el esfuerzo*: de la escritura hacia la **especificación**, la **revisión** y la **validación**.

Esta observación es una **Hipótesis** de trabajo, no un resultado medido. El AI-SDLC añade una tarea que el desarrollo clásico no siempre nombra: la **selección y gobierno del sistema de IA**.

Etiquetado epistémico inicial

- **Evidencia:** revisión, trazabilidad y verificación de requisitos son prácticas consolidadas [8, 9, 4].
- **Propuesta docente:** la secuencia de fases, sus entradas, responsables, evidencias y salidas; la tabla resumen y el modelo de gobernanza.
- **Hipótesis:** que seguir esta secuencia reduzca el riesgo en un contexto dado (no medido).

Desarrollo guiado por especificaciones (SDD)

- Dirigir la implementación solo por lenguaje natural tiene límites conocidos: instrucciones ambiguas o contradictorias, pérdida de contexto, sin traza de qué se pidió, difícil de evaluar.
- El SDD propone artefactos explícitos (requisitos, planificación, tareas, criterios de aceptación) que median entre la intención y la implementación.
- La aportación contemporánea no es especificar (antiguo), sino la especificación como mecanismo de **coordinación y control** del trabajo de agentes de IA.

Propuesta metodológica: estructura el proceso, no garantiza por sí sola el resultado.

SDD y agentes: fundamentos y límites

- Dos sentidos de «especificación»: en *lenguaje natural estructurado* (p. ej. notación EARS de Hello SDD [7]) y *formal* (contratos verificables).
- Una especificación no es completa, correcta ni verificable por estar documentada.
- Las especificaciones son *artefactos de coordinación*, pero **no** sustituyen la verificación independiente: compartir especificación no hace independientes a los agentes.
- Implementaciones: GitHub Spec Kit [3] (constitución + etapas) y Hello SDD [7] (flujo docente ligero con EARS). Son documentación de prácticas, no estándares.
- **SDD** (estructura el desarrollo) $\neq$ **Verified Software Engineering** (garantías mediante verificación formal, pruebas y análisis).

Tres aproximaciones al mismo caso

Aproximación	Qué produce	Qué no garantiza
A. Prompt-driven	Código a partir de instrucciones directas	Trazabilidad, cobertura, evidencias
B. Specification-driven	Requisitos, plan, tareas, implementación y pruebas estructuradas	Que la especificación sea correcta; garantías formales
C. Ciclo de ingeniería asistido	B + verificación con evidencias, aprobación, operación, reversibilidad	Que las pruebas sean exhaustivas; corrección general

Describe qué produce cada enfoque, *no* una jerarquía de calidad. **Propuesta docente**.

Cuatro modalidades de integración de la IA en el SDLC

SDLC sin IA generativa. Incluye la automatización convencional (CI, pruebas, SAST, ágil, DevOps); la IA se incorpora a ese sustrato, no lo sustituye.

SDLC asistido por IA. La IA asiste en actividades concretas dentro de un proceso preexistente.

SDLC nativo de IA. El proceso se reorganiza en torno a la IA: artefactos, puertas y coordinación se diseñan asumiendo su participación continua [2].

Desarrollo agéntico. Agentes ejecutan secuencias de acciones con herramientas y entornos [2].

No es una clasificación normalizada ni una escala de madurez; son distinciones conceptuales orientativas (**Propuesta docente**). La IA desplaza el esfuerzo y añade riesgos; no sustituye la verificación, la aprobación ni la responsabilidad.

Modalidades: dimensiones

Dimensión	Sin IA generativa	Asistido por IA	Nativo/agéntico
Organización	Cascada, ágil, DevOps	Proceso existente + herramientas IA	Proceso adaptado a IA sistemática
Requisitos	Elaboración humana	IA propone y analiza	Agentes transforman especificación en tareas
Implementación	Humana y automatización	Generación asistida	Ejecución autónoma de tareas
Pruebas	Automáticas o manuales	IA propone pruebas	Ciclos generación-prueba-corrección
Revisión	Humana y herramientas	Humana apoyada por IA	Multinivel con controles independientes
Riesgo adicional	Convencionales	Confianza no calibrada	Acciones autónomas, permisos, contexto

Autonomía de ejecución vs. autoridad de aprobación

Capacidad técnica. Lo que el agente *puede* hacer con herramientas y permisos.

Permiso. Lo que el sistema *le deja* hacer según credenciales y política de acceso.

Autoridad. Lo que está *legítimamente facultado* a decidir (aprobar, aceptar riesgo).

Responsabilidad. A quién corresponde *responder* por el resultado y el riesgo residual.

La autonomía de ejecución no implica autonomía de aprobación. Un agente puede generar, probar y proponer sin estar autorizado a aprobar su propio trabajo, aceptar riesgo residual, cambiar políticas o desplegar. La aprobación sigue siendo humana.

Las diez fases del AI-SDLC (1-5)

F.	Fase	Objetivo	Salida principal
1	Requisitos funcionales	Declarar qué debe hacer y bajo qué supuestos	Especificación funcional verificable
2	Requisitos de seguridad	Declarar qué no violar y qué se protege	Especificación de seguridad y amenazas
3	Plataforma	Fijar entorno de desarrollo y ejecución	Plataforma y herramientas registradas
4	Selección del sistema de IA	Elegir y registrar la tupla	Ficha de la tupla
5	Generación	Producir el artefacto candidato	Artefacto + condiciones registradas

Las diez fases del AI-SDLC (6-10)

F.	Fase	Objetivo	Salida principal
6	Revisión	Inspeccionar antes de validar	Informe y hallazgos clasificados
7	Validación	Comprobar requisitos con el pipeline	Expediente de evidencia por requisito
8	Atestación del aislamiento	Verificar y registrar el aislamiento usado	Registro de configuración y evidencia
9	Aceptación	Decidir la promoción con evidencia	Decisión registrada y firmada
10	Mantenimiento	Sostener el artefacto ante el cambio	Procedimiento y trazabilidad continuada

Orden de dependencia, no cadena rígida. Las fases *no* son un estándar ni un certificado (**Propuesta docente**).

Fases 1-4: del requisito a la selección

- **F1 - Requisitos funcionales:** leer CSV, normalizar, generar PDF/XLSX; cada requisito identificado y comprobable. Decidir *qué se necesita* es tarea humana.
- **F2 - Requisitos de seguridad:** no confiar en el CSV, evitar *path traversal*, temporales seguros, límites de tamaño/tiempo, no ejecutar contenido embebido, neutralizar fórmulas (CWE [6], CERT [10]). Es una *propiedad a verificar*, no un certificado.
- **F3 - Plataforma:** SO, arquitectura, intérprete, dependencias fijadas, imagen por resumen criptográfico. Reproducibilidad y aislamiento son propiedades distintas.
- **F4 - Selección del sistema de IA:** no «un modelo», sino la *tupla* (modelo, versión, configuración, *prompt*/contexto, herramientas, entorno).

La responsabilidad humana no se delega: el registro de la tupla es la condición para responder de *qué* se usó, *cómo* y con *qué* consecuencias.

Fases 5-8: generar, revisar, validar, atestar

- **F5 - Generación:** el artefacto es una *hipótesis de solución*; registrar las condiciones. Código generado no es código verificado.
- **F6 - Revisión:** revisión humana estructurada (formato, dependencias, supuestos, legibilidad); hallazgos clasificados y trazables a un requisito.
- **F7 - Validación:** pipeline del capítulo 10; **toda etapa que ejecute código no fiable dentro del aislamiento**; producto = expediente por requisito, no veredicto.
- **F8 - Atestación del aislamiento:** el aislamiento es *requisito previo y transversal* a las actividades de F7; se registra y audita la configuración usada.

Expediente verde $\neq$ sin vulnerabilidades: hereda los falsos negativos de cada técnica.

Fases 9-10: aceptar y mantener

- **F9 - Aceptación:** la decisión no se deduce de un expediente verde; la *asume* una persona informada. Formulación honesta: «bajo las comprobaciones A, B y C, en el entorno descrito, no se observaron fallos; se aceptan los riesgos X e Y».
- **F10 - Mantenimiento:** actualizar dependencias, repetir la validación cuando cambie algo relevante, conservar trazabilidad. Cada formato o dependencia amplía la superficie de ataque.
- El equilibrio KISS/YAGNI es una decisión de seguridad: lo que no existe no puede ser vulnerable.

La comprobación de las puertas y del aislamiento (prueba de rechazo) forma parte de la validez del propio proceso: un fallo de infraestructura o una prueba omitida no deben convertirse en aprobado.

De la aceptación a la operación

- El ciclo no termina en la aceptación: el software se despliega y produce *evidencia operacional*.
- La evidencia operacional responde a «¿se comporta como se esperaba con datos y dependencias reales?»; puede confirmar supuestos, revelar defectos o exigir suspender el cambio.
- La operación es una fuente de nueva evidencia que realimenta el proceso.

La operación *no* demuestra corrección: la ausencia de incidentes hereda los puntos ciegos de la observabilidad.

Capacidades operativas y límites

Capacidad	Qué permite	Límite a declarar
Feature flags	Activar/desactivar sin re-desplegar	Añaden estados y combinaciones no probadas
Despliegue gradual	Exponer a un subconjunto antes que al total	Reduce el alcance, no la probabilidad
Observabilidad	Detectar anomalías y regresiones	La ausencia de señales no demuestra ausencia
Rollback / desactivación	Retirar una versión problemática	No garantiza recuperar servicio ni estado
Restauración de estado	Recuperar datos o estado previo	No siempre reproduce el estado exacto
Efectos externos	-	Algunos son irreversibles (datos a terceros)

Los mecanismos de recuperación deben *probarse*: un rollback no ensayado es una hipótesis, no una capacidad (**Propuesta docente**).

Gobernanza: cinco ámbitos

Ámbito	Decisión de gobernanza
Herramientas autorizadas	Qué asistentes, analizadores y servicios se permite usar, dónde y con qué configuración
Datos y confidencialidad	Qué datos pueden ir a un *prompt* o servicio externo; minimización y anonimización
Licencias	Licencias del código generado y dependencias; compatibilidad y atribución
Credenciales	Gestión de claves y secretos; qué credenciales ve el asistente; rotación y alcance mínimo
Responsabilidad humana	Quién responde de cada decisión; revisión humana obligatoria; registro

La gobernanza no sustituye a la técnica ni al contrario. Datos en el *prompt* y confidencialidad es una decisión de gobernanza y cumplimiento, adoptada *antes* de usar el sistema.

Del método a la operación: AG-00 y P00

- El kit `kit_agentes_ai_sdlc_v2` traduce la metodología: `AG-00_orchestrator` coordina roles y `P00_integral` recorre la secuencia completa.
- Principio transversal: **independencia entre generación y evaluación** (el generador no aprueba su propio resultado, ni cambia criterios, ni modifica pruebas protegidas).
- Controles mínimos: separar permisos de escritura; fijar criterios y pruebas antes de generar; registrar versión, comandos, herramientas y resultados; bloquear si faltan evidencias; decisión final de un rol distinto del generador.

Modelos distintos no son independencia: la independencia es propiedad del proceso, no del proveedor. La secuencia es **Propuesta docente**, no estándar ni certificación.

Coste total conceptual

Componente	Qué incluye y cómo lo afecta la IA
Implementación	Especificar, diseñar, escribir. La IA reduce escritura y andamiaje, pero añade formular buenos *prompts* y gobernar el sistema
Verificación	Revisar, validar, construir evidencia. Tiende a *aumentar* en términos relativos
Mantenimiento	Actualizar dependencias, repetir validaciones, conservar trazabilidad. Crece con cada añadido
Riesgo	Daño esperado. No desaparece: la IA puede introducir superficie nueva; el riesgo residual se declara y acepta

Marco de razonamiento (**Propuesta docente** + **Hipótesis**), *no* una ley empírica. No se afirman magnitudes ni proporciones.

Límites de la metodología

- El AI-SDLC **no es un estándar oficial ni una certificación**.
- La alineación con NIST SSDF, OWASP ASVS, CWE, CERT e ISO/IEC 27034 es *conceptual*: reutiliza ideas, no las reproduce ni está reconocida por ellos.
- Superar las fases no demuestra seguridad: el invariante se mantiene en cada fase.
- El coste total es un marco conceptual, no una medición.
- Los ejemplos vulnerables solo se ejecutan en aislamiento autorizado.
- Las correspondencias concretas con cada marco deben justificarse por identificador y versión.

Síntesis del capítulo

- La IA *desplaza* el esfuerzo hacia especificación, revisión y validación; no elimina los problemas ni garantiza seguridad.
- El SDD estructura el proceso con artefactos explícitos; no garantiza corrección por sí mismo.
- Cuatro modalidades de integración (sin IA, asistida, nativa, agéntica): distinción conceptual, no escala de madurez.
- Diez fases con *orden de dependencia*; la autonomía de ejecución no implica autoridad de aprobación.
- Gobernanza en cinco ámbitos; el método es una propuesta docente, no un estándar.

Preguntas de reflexión

1. ¿Por qué el AI-SDLC insiste en que no es un estándar oficial? ¿«Alinearse» vs. «ser conforme»?
2. ¿Por qué la selección del sistema de IA se define como una *tupla* y no «un modelo»?
3. «Como el pipeline está verde, puedo aceptar sin revisar»: ¿qué fases se ignoran y por qué es un error?
4. ¿Por qué el coste de verificación tiende a aumentar cuando la IA escribe más código?
5. ¿Qué inconsistencia aparece si las pruebas de F7 se ejecutan fuera del aislamiento de F8?

¿Qué responsabilidades no se delegan?

- ¿Qué responsabilidades humanas no pueden delegarse al sistema de IA en ninguna fase?

Referencias I



Joel Becker, Nate Rush, Beth Barnes, and David Rein.

Measuring the impact of early-2025 ai on experienced open-source developer productivity.

Informe experimental, Model Evaluation & Threat Research (METR), July 2025.

Estudio contextualizado (RCT con desarrolladores experimentados, 2025); halló una ralentización en su contexto; no es un efecto universal.



Endor Labs.

What is the ai-native sdlc? ai sdlc, ai-dlc, and adlc explained.

Guía técnica (Learn), publicada el 22 de octubre de 2026 y actualizada el 27 de octubre de 2026, 2026.

Documentación técnica de un proveedor; no es literatura científica revisada.



GitHub.

Spec kit: toolkit for specification-driven development (sdd).

Documentación técnica / propuesta metodológica. Consultado en noviembre de 2026.

Referencias II

ISO/IEC.
Iso/iec 27034 — seguridad de la información: Seguridad de las aplicaciones.
Norma; debe citarse la parte y edición específicas al aplicar requisitos.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kelly Pei, Ofir Press, and Karthik Narasimhan.
Swe-bench: Can language models resolve real-world github issues?
In *International Conference on Learning Representations (ICLR)*, 2024.
Publicación científica; superar sus pruebas no demuestra ausencia de vulnerabilidades.

MITRE Corporation.
Common weakness enumeration (cwe).
Catálogo de referencia; los identificadores deben justificarse por caso.

Brais Moure.
Hello sdd: curso de spec-driven development (sdd) desde cero.
Documentación técnica / material docente. Consultado en noviembre de 2026.

Referencias III



National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.



OWASP Foundation.

Application security verification standard (asvs), 2025.

Guía de referencia (versión estable 5.0.0, 2025).



SEI / CERT Division, Carnegie Mellon University.

Sei cert c coding standard.

Technical report, 2016.

Edición CERT C 2016.